

A small app, on your iPhone

A practical guide to logging, keeping your data safe, and understanding how a website becomes a personal app.

Daily Check-in is a progressive web app (PWA): an interactive website that you can add to your Home Screen. You do not compile anything on your iPhone. GitHub builds the code and serves the app; your phone runs it.

Install once, then open the icon

1. Open [Daily Check-in in Safari](#) while connected to the internet.
2. Tap **Share**, then **Add to Home Screen**. If available, enable **Open as Web App**. Tap **Add**.
3. Open the new **Daily Check-in** icon. Let the first online load finish so the app can prepare its offline files.
4. Use the installed app consistently. Safari and an installed web app may use different data containers.

No MacBook needed

For this app, a browser, GitHub repository, and hosting are enough. The app installs from a web link rather than the App Store.

What is included

One editable daily record for perceived intake, weight, and optional cardio. Trends help you review what you have logged. History lets you revisit earlier dates. Settings provides CSV export and JSON backup/restore.

App: [Open Daily Check-in](#)

Source and version history: [GitHub repository](#)

Installation reference: [Apple: Turn a website into an app in Safari](#)

A check-in takes a moment

The date defaults to today in Hong Kong time. You can record a missed day, but future dates are not accepted.

Your daily fields

Perceived intake (1-5) is your impression: very low, low, usual, high, or very high. It is not a calorie calculation. **Weight** is entered in kilograms.

Cardio: choose Yes or No. If Yes, enter your own cardio type, minutes, and burned calories. The type can be walking, cycling, swimming, or any text that makes sense to you. Numbers are manually entered; the app does not measure activity or calculate calories.

Example for 6 October

Intake: 3 (usual) | Weight: 70.2 kg
Cardio: Yes | Type: Walking
Minutes: 30 | Burned calories: 180 kcal

The 180 kcal value is an example input, not an estimate supplied by this app.

Saving and correcting

Tap **Save check-in**. A date has one record: saving it again replaces that date's existing entry. Use **History** → **Edit** to change it. Blank fields mean not recorded; an explicit cardio No means you recorded no cardio.

You can save an intake, weight, or cardio answer without filling every field. A cardio No can therefore be a useful check-in on its own. Previous records remain valid after the cardio update.

Reading trends

Use the selected time range to review changes and logged days. Missing observations stay missing rather than becoming zero. The weight average uses the available weights within seven calendar days. Cardio summaries reflect only your entered values, so partial logs are a partial picture.

Where does the journal live?

The public code repository and your private daily records are two different things.

When you tap Save, JavaScript validates the record and writes it to **IndexedDB**, a database managed by your browser on your phone. This project calls its database **daily-check-in**. It has an **entries** store keyed by date, plus a **settings** store for preferences.

What happens when you return?

The app opens the same local database, reads the entries, and draws the history and graphs again. Closing and reopening the app normally preserves these saved records.

What leaves the phone?

The current journal implementation does not upload your intake, weight, or cardio records to GitHub or Supabase. GitHub supplies the app files; it does not act as your journal database. There is no account-based journal sync between phones.

The optional reminder service, once configured, receives a push subscription and reminder settings. That is separate from your daily records. Its backend setup and delivery have not been verified as complete.

The app does not encrypt journal records itself or protect them with a separate password. Local storage is useful for keeping logs off the server, but it is not a guarantee against access by someone using your unlocked phone or trusted code running at the same origin.

Local does not mean permanent

Browser storage can be cleared by you or removed under storage pressure. Deleting/reinstalling the app, changing phone, or changing its hosting address can lose access to the records. Private browsing is unsuitable for a lasting journal. Keep a JSON backup somewhere outside the app.

Storage is scoped to an **origin**: protocol + hostname + port. The path alone is not a security boundary. Other trusted web code hosted on the same origin may access its databases. Keep control of the GitHub account and the code deployed there.

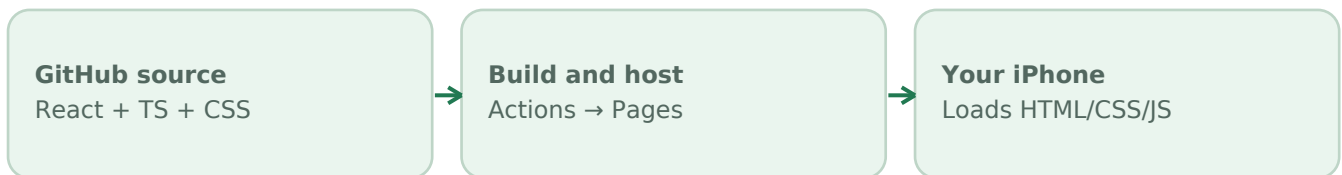
References: [MDN: Using IndexedDB](#); [MDN: Storage quotas and eviction](#)

How HTML becomes an app

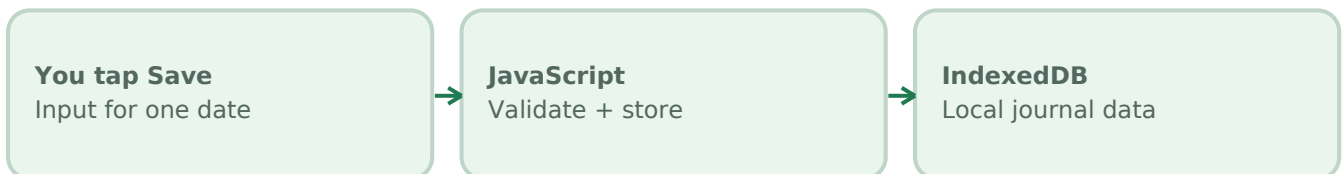
HTML supplies the starting document; JavaScript supplies the interactive behavior.

HTML is the page skeleton and loads the app files. **CSS** controls colors, spacing, and responsive layout. **JavaScript** handles input, saving, exporting, and charts. This app uses React to construct the interface; Vite converts its source into deployable static files.

The two flows



ONLINE DELIVERY: files travel to the phone. Your daily records are not part of this upload.



LOCAL JOURNAL: data stays in the phone's browser/app container.

Why can it run offline?

A **service worker** is a browser-managed helper. After a successful online installation, this app caches its HTML, JavaScript, CSS, icons, and fonts. Later it can load those files without downloading them again. This code cache is separate from the IndexedDB journal.

Offline use depends on that initial cache being ready and retained. The service worker is not a program that can run continuously in the background; the browser controls when it starts and stops.

Reference: [MDN: Offline and background operation](#)

Protect your entries

Choose the right export for what you want to do next.

CSV: for spreadsheets and analysis

In **Settings** → **Export spreadsheet (CSV)**, save or share the file using the iPhone share sheet. Open it in Numbers, Excel, or Google Sheets. Each row is a date; columns include intake level, weight, cardio Y/N, type, minutes, and calories. Empty cells mean not recorded.

CSV is an analysis export. This app does not restore from CSV. If you edit spreadsheet values, those changes do not automatically flow back into your journal.

JSON: for backup and restoration

Use **Settings** → **Export backup** and save the JSON file to Files, iCloud Drive, or another safe location. This backup contains entries, not your reminder subscription or settings. Keep dated copies, especially before deleting data, replacing your phone, or changing the app address.

On the destination phone/app, choose **Restore from backup**, select the JSON file, review the preview, and confirm. Incoming dates replace matching dates; other dates remain. Version 1 backups from the older app are still accepted. New version 2 backups include cardio fields.

A useful routine

Once a week: export a JSON backup.
Before a risky change: export another backup.
For charts outside the app: export CSV too.
Check that Files really contains the saved files.

How the app updates

A push to GitHub's main branch runs checks, builds the files, and deploys them to Pages. When you next go online, your browser can discover the new service worker. Save your current form, close all app/Safari windows for this app, and reopen to let the waiting update activate.

An update at the same address does not intentionally erase the local journal. The cardio release accepts older records without rebuilding the database. Still, backup first: hosting changes and browser storage loss need explicit recovery from your JSON file.

Is this a useful way forward?

Yes - for small personal tools with forms, local records, exports, and charts, a PWA is a practical approach.

You can reuse the same pattern for a habit diary, expense log, reading tracker, symptom notes, or study planner: define a record, build an interface, save it locally, add export/restore, then host the static files. You can develop on Windows or Linux and install through Safari without a Mac or App Store release.

Choose the approach around the features

Need	This PWA approach	Native iOS app
Manual logs and charts	Good fit; offline storage	Also suitable
Install and update	Web link + Home Screen	Apple build/distribution process
Cross-device journal sync	Add a backend and accounts	Needs a sync design too
Apple Health / continuous tracking	Not provided by this app	Native APIs may be needed

This app does not read Apple Health, GPS workouts, or heart rate, and does not continuously measure exercise in the background. Burned calories are your input. Reliable scheduled push reminders require the separate backend plus device permission and testing; a cached page alone cannot guarantee a daily alert.

A simple development recipe

1. Start with the smallest useful form and one clear data model.
2. Decide whether local-only data is enough; add backups early.
3. Test saving, reopening, offline use, and restoration on the actual phone.
4. Publish a version, keep its history in GitHub, and make small updates.

Further reading: [MDN: What is a progressive web app?](#)

This guide describes the project implementation at v1.1.0. iPhone menus and browser capabilities can vary with iOS versions. Use the linked primary documentation for current platform guidance.